

UCS2612 Machine Learning Laboratory
Academic Year: 2023-2024
Even
Faculty In-charges: Y.V. Lokeswari & Nilu R Salim A.
No. : 3
Batch: 2021-2025 VI Semester A & B

Assignment on PLA with User Defined Functions

Aathish Sivasubrahmanian
3122215001001

Objective:

Develop a python program to make predictions for the iris dataset using PLA.

CODE:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.base import BaseEstimator, ClassifierMixin
import matplotlib.pyplot as plt

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Choose two classes for binary classification
class1 = 0 # Change this to choose the class you want
class2 = 1 # Change this to choose the class you want

# Filter dataset to keep only samples of the chosen classes
X = X[(y == class1) | (y == class2)]
y = y[(y == class1) | (y == class2)]

# Map class labels to -1 and 1
y[y == class1] = -1
y[y == class2] = 1

# Add bias term to input data
X = np.c_[np.ones(X.shape[0]), X]
```

```

# Perceptron Learning Algorithm
class Perceptron(BaseEstimator, ClassifierMixin):
    def __init__(self, learning_rate=0.1, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1]) # Initialize weights
        for epoch in range(self.epochs):
            for i in range(X.shape[0]):
                if y[i] * np.dot(X[i], self.w) <= 0:
                    self.w += self.learning_rate * y[i] * X[i]
            return self

    def predict(self, X):
        return np.sign(np.dot(X, self.w))

# Cross-validation
perceptron = Perceptron()
cv_scores = cross_val_score(perceptron, X, y, cv=5)
print("Cross-Validation Scores:", cv_scores)
print("Average Cross-Validation Score:", np.mean(cv_scores))

from sklearn.linear_model import Perceptron
from sklearn.model_selection import cross_val_score
import numpy as np

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Choose two classes for binary classification
class1 = 0 # Change this to choose the class you want
class2 = 1 # Change this to choose the class you want

# Filter dataset to keep only samples of the chosen classes
X = X[(y == class1) | (y == class2)]
y = y[(y == class1) | (y == class2)]

# Map class labels to -1 and 1
y[y == class1] = -1
y[y == class2] = 1

```

```
# Perform cross-validation with scikit-learn's Perceptron
perceptron_sklearn = Perceptron()
cv_scores_sklearn = cross_val_score(perceptron_sklearn, X, y, cv=5)
print("Cross-Validation Scores (scikit-learn Perceptron):", cv_scores_sklearn)
print("Average Cross-Validation Score (scikit-learn Perceptron):", np.mean(cv_scores_sklearn))
```

OUTPUT:

User defined:

```
# Cross-validation
perceptron = Perceptron()
cv_scores = cross_val_score(perceptron, X, y, cv=5)
print("Cross-Validation Scores:", cv_scores)
print("Average Cross-Validation Score:", np.mean(cv_scores))
```

1] ✓ 1.1s

· Cross-Validation Scores: [1. 1. 1. 1. 1.]
Average Cross-Validation Score: 1.0

[+ Code](#) [+](#)

Built in PLA:

```
# Cross-validation
perceptron = Perceptron()
cv_scores = cross_val_score(perceptron, X, y, cv=5)
print("Cross-Validation Scores:", cv_scores)
print("Average Cross-Validation Score:", np.mean(cv_scores))
```

1] ✓ 1.1s

· Cross-Validation Scores: [1. 1. 1. 1. 1.]
Average Cross-Validation Score: 1.0

[+ Code](#) [+](#)

Inference:

- We can infer from the above results that PLA performs well in iris dataset with 100% accuracy
- We can also infer that both built in and user defined PLA gives the same accuracy.

Learning Outcomes:

- Understood and implemented the concept of PLA .
- Understood and implemented user defined custom designed models for PLA and compared the results with the built in functions.